

02 — Hashing

DSA in Python — Interview Prep

Hashing — Tutorial

Covers the 10 problems solved in this folder. Every problem here uses a dict or set to turn an $O(n)$ or $O(n^2)$ lookup into $O(1)$ amortized — that trade of memory for speed is the entire point of this topic.

The patterns

1. Complement lookup (01)

Walking once, and for each element asking "have I already seen the value that would pair with this one to hit my target?" Store value $\rightarrow$ index as you go; check before you insert so you never match an element with itself.

```
seen = {}
for i, n in enumerate(nums):
    if (target - n) in seen:
        return [seen[target - n], i]
    seen[n] = i
```

Turns the brute-force $O(n^2)$ pair check into a single $O(n)$ pass.

2. Plain membership (02, 08)

When the question is just "have I seen this before" or "what's common between two collections," a **set** alone is enough — no need for a dict with values attached.

- **contains_duplicate**: add-then-check as you scan.
- **intersection_of_two_arrays**: two sets, **&** operator. Sometimes the "hashing technique" *is* just knowing the right built-in operator exists and is already optimal.

3. Frequency counting (03, 05, 10)

Build a dict of value $\rightarrow$ count, then use those counts to answer a question. Three different questions, same counting step:

- **valid_anagram**: build counts from s, **decrement** while consuming t — a clean way to check two multisets are equal without building two separate dicts and comparing them.
- **top_k_frequent_elements**: counts feed into a bucket-sort indexed by frequency (bucket index = frequency value, since frequency is bounded by **len(nums)**), giving $O(n)$ instead of the $O(n \log n)$ a sort-by-frequency approach would cost.
- **first_unique_character**: counts first, then a **second** pass in original order to find the first count-1 entry — needed because "first" refers to original string order, not count order.

4. Canonical-key grouping (04)

When items need to be bucketed by "sameness" under some transformation, compute a canonical form and use it as a dict key. **group_anagrams** sorts each string's characters — every anagram of a word produces the

identical sorted string, so that sorted string is a perfect grouping key. This generalizes: any "group things that are equivalent under transformation X" problem reduces to "compute X, use it as a dict key."

5. Prefix-sum + hash (07)

The hardest pattern here. When you need to count subarrays matching a sum condition, and the array has negative numbers (so sliding window's "shrink when too big" logic breaks), track running prefix sums in a dict of sum -> how many times seen. A subarray from j+1 to i sums to k exactly when `prefix[i] - prefix[j] == k`, so at each step you look up `prefix[i] - k` in the dict of prior prefix sums. Seeding with `{0: 1}` is required to count subarrays that start at index 0 (their "j" is the empty prefix before the array starts).

6. Bijective (two-way) mapping (09)

Some problems need *both directions* of a mapping to stay consistent, not just one. `isomorphic_strings` keeps two dicts (s->t and t->s) because a single dict would silently allow two different source characters to map to the same target character, which breaks the one-to-one requirement. Whenever "mapping" and "no two things collide into the same target" both appear in a problem statement, that's the signal to keep both directions.

7. Set-based sequence detection (06)

`longest_consecutive_sequence` looks like a sorting problem but sorting costs $O(n \log n)$ and the problem asks for $O(n)$. The trick: put everything in a set, then only *start* counting a run from a number whose predecessor ($n-1$) is not in the set. That guarantees every run is walked exactly once, from its true start — total work across all runs is $O(n)$, not $O(n)$ per element.

Complexity summary

#	Problem	Time	Extra space
01	two_sum	$O(n)$	$O(n)$
02	contains_duplicate	$O(n)$	$O(n)$
03	valid_anagram	$O(n)$	$O(1)$ (bounded alphabet)
04	group_anagrams	$O(n \cdot k \log k)$	$O(n \cdot k)$
05	top_k_frequent_elements	$O(n)$	$O(n)$
06	longest_consecutive_sequence	$O(n)$	$O(n)$
07	subarray_sum_equals_k	$O(n)$	$O(n)$
08	intersection_of_two_arrays	$O(n+m)$	$O(n+m)$
09	isomorphic_strings	$O(n)$	$O(1)$ (bounded alphabet)
10	first_unique_character	$O(n)$	$O(1)$ (bounded alphabet)

(k in 04 is average string length, from the per-string sort.) Unlike topic 01, this whole topic trades $O(n)$ *extra space* for dropping time complexity from $O(n^2)/O(n \log n)$ down to $O(n)$ — that trade is the defining feature of hashing problems, versus topic 01's in-place $O(1)$ -space focus.

Common interview follow-ups to be ready for

- `two_sum`: "what if there's no valid pair?" — returning `[]` here; confirm with the interviewer whether that's acceptable or an exception/sentinel is expected instead.

- `two_sum`: "what if the array is sorted?" — then two-pointer from both ends beats hashing on space ($O(1)$ instead of $O(n)$); know when hashing is **not** the best tool.
- `group_anagrams`: "what if strings can have unicode / very long length?" — sorting-as-key costs $O(k \log k)$ per string; a character **count tuple** as the key avoids the sort, dropping to $O(k)$ per string at the cost of a bulkier key.
- `top_k_frequent_elements`: "what if k is close to n ?" — bucket sort here is still $O(n)$ regardless of k , unlike a heap-based $O(n \log k)$ approach; know why bucket sort wins whenever frequency is bounded.
- `subarray_sum_equals_k`: "what if you need the actual subarrays, not just the count?" — the dict would need to store lists of indices per prefix sum instead of just counts.
- `longest_consecutive_sequence`: "why not just sort?" — be ready to state the $O(n \log n)$ vs $O(n)$ distinction explicitly; sorting is the "obvious" answer interviewers expect you to beat.
- `isomorphic_strings`: "what about three or more strings needing the same isomorphism?" — the two-dict trick generalizes to checking pairwise consistency across all of them.

Self-check before moving on

You should be able to, without looking at the code:

- Explain why `two_sum`'s dict check happens **before** inserting the current element.
- Derive the prefix-sum equation for `subarray_sum_equals_k` on paper (why `prefix[i] - prefix[j] == k`, and why `{0: 1}` is seeded).
- Explain why `longest_consecutive_sequence` doesn't just sort first.
- State from memory when a plain dict is enough vs when you need two dicts (bijective mapping) vs when you need a set instead of a dict.

If any of those aren't fluent yet, this topic isn't "done" per the README's definition — revisit before moving to `03_two_pointers_sliding_window`.

Programs

01. Two Sum

01_two_sum.py

Explanation

Problem: Given an array of integers `nums` and an integer `target`, return the indices of the two numbers that add up to target. Exactly one solution exists; can't use the same element twice.

Source : LeetCode 1 - Two Sum

Example:

Input: `nums = [2, 7, 11, 15]`, `target = 9`

Output: `[0, 1]` (`2 + 7 == 9`)

Idea: brute force is $O(n^2)$ checking every pair. Instead, walk once and for each number ask "have I already seen its complement (`target - n`)?" via a dict — turns the lookup into $O(1)$ amortized.

Code

```
def two_sum(nums: list[int], target: int) -> list[int]:
    seen = {} # value -> index

    for i, n in enumerate(nums):
        complement = target - n
        if complement in seen:
            return [seen[complement], i]
        seen[n] = i

    return []

if __name__ == "__main__":
    tests = [
        ([2, 7, 11, 15], 9, [0, 1]),
        ([3, 2, 4], 6, [1, 2]),
        ([3, 3], 6, [0, 1]),
    ]

    for i, (nums, target, expected) in enumerate(tests, 1):
        got = two_sum(nums, target)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

02. Contains Duplicate

02_contains_duplicate.py

Explanation

Problem: Given an integer array `nums`, return True if any value appears at least twice, False if every element is distinct.

Source : LeetCode 217 - Contains Duplicate

Example:

Input: [1, 2, 3, 1]

Output: True

Idea: a set gives $O(1)$ average membership checks. Walk once; if the current element is already in the set, a duplicate has been found.

Code

```
def contains_duplicate(nums: list[int]) -> bool:
    seen = set()

    for n in nums:
        if n in seen:
            return True
        seen.add(n)

    return False

if __name__ == "__main__":
    tests = [
        ([1, 2, 3, 1], True),
        ([1, 2, 3, 4], False),
        ([1, 1, 1, 3, 3, 4, 3, 2, 4, 2], True),
        ([], False),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        got = contains_duplicate(inp)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

03. Valid Anagram

03_valid_anagram.py

Explanation

Problem: Given two strings *s* and *t*, return True if *t* is an anagram of *s* (same characters, same counts, any order).

Source : LeetCode 242 - Valid Anagram

Example:

Input: *s* = "anagram", *t* = "nagaram"

Output: True

Idea: build a frequency count of *s*, then walk *t* decrementing counts.

If *t* ever needs a character that's not available (missing or count already 0), it's not an anagram. If the count dict is empty at the end, every character balanced out exactly.

Code

```
def is_anagram(s: str, t: str) -> bool:
```

```

    if len(s) != len(t):
        return False

    counts = {}
    for ch in s:
        counts[ch] = counts.get(ch, 0) + 1

    for ch in t:
        if ch not in counts:
            return False
        counts[ch] -= 1
        if counts[ch] == 0:
            del counts[ch]

    return len(counts) == 0

if __name__ == "__main__":
    tests = [
        ("anagram", "nagaram", True),
        ("rat", "car", False),
        ("a", "ab", False),
        ("", "", True),
        ("aacc", "ccac", False),
    ]

    for i, (s, t, expected) in enumerate(tests, 1):
        got = is_anagram(s, t)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```

04. Group Anagrams

04_group_anagrams.py

Explanation

Problem: Given an array of strings, group the anagrams together. Return the groups in any order (order within a group also doesn't matter for grading here).

Source : LeetCode 49 - Group Anagrams

Example:

Input: ["eat", "tea", "tan", "ate", "nat", "bat"]

Output: [["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]

Idea: anagrams share the same character multiset, so sorting a string's characters gives a canonical key that's identical for every anagram of it. Use that sorted string as a dict key and bucket originals under it.

Code

```

def group_anagrams(strs: list[str]) -> list[list[str]]:
    groups: dict[str, list[str]] = {}

    for s in strs:
        key = "".join(sorted(s))
        groups.setdefault(key, []).append(s)

```

```

return list(groups.values())

def _normalize(groups: list[list[str]]) -> list[list[str]]:
    return sorted(sorted(g) for g in groups)

if __name__ == "__main__":
    tests = [
        (["eat", "tea", "tan", "ate", "nat", "bat"],
         [[["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]]),
        ([[], [[]]]),
        (["a"], [["a"]]),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        got = group_anagrams(inp)
        status = "PASS" if _normalize(got) == _normalize(expected) else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```

05. Top K Frequent Elements

05_top_k_frequent_elements.py

Explanation

Problem: Given an integer array `nums` and integer `k`, return the `k` most frequent elements.

Source : LeetCode 347 - Top K Frequent Elements

Example:

Input: `nums = [1, 1, 1, 2, 2, 3]`, `k = 2`

Output: `[1, 2]`

Idea: sorting all (value, freq) pairs by frequency is $O(n \log n)$.

Bucket sort does it in $O(n)$: frequency can never exceed `len(nums)`, so make a list of buckets indexed by frequency, drop each value into `bucket[its frequency]`, then read buckets from highest frequency down until `k` values are collected.

Code

```

def top_k_frequent(nums: list[int], k: int) -> list[int]:
    counts: dict[int, int] = {}
    for n in nums:
        counts[n] = counts.get(n, 0) + 1

    buckets: list[list[int]] = [[] for _ in range(len(nums) + 1)]
    for num, freq in counts.items():
        buckets[freq].append(num)

    result = []
    for freq in range(len(buckets) - 1, 0, -1):
        for num in buckets[freq]:
            result.append(num)
            if len(result) == k:
                return result

    return result

```

```

if __name__ == "__main__":
    tests = [
        ([1, 1, 1, 2, 2, 3], 2, [1, 2]),
        ([1], 1, [1]),
        ([4, 4, 4, 6, 6, 6, 1], 2, [4, 6]),
    ]

    for i, (nums, k, expected) in enumerate(tests, 1):
        got = top_k_frequent(nums, k)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```

06. Longest Consecutive Sequence

06_longest_consecutive_sequence.py

Explanation

Problem: Given an unsorted array of integers, return the length of the longest run of consecutive integers (order in the array doesn't matter). Must run in $O(n)$ time.

Source : LeetCode 128 - Longest Consecutive Sequence

Example:

Input: [100, 4, 200, 1, 3, 2]

Output: 4 (the sequence 1, 2, 3, 4)

Idea: put everything in a set for $O(1)$ membership checks. Only start counting a sequence from a number that is the *start* of one, i.e. $n - 1$ is not in the set. That guarantees each sequence is only ever walked once in full, from its start, giving $O(n)$ total work instead of $O(n)$ per starting point.

Code

```

def longest_consecutive(nums: list[int]) -> int:
    num_set = set(nums)
    longest = 0

    for n in num_set:
        if n - 1 not in num_set:
            length = 1
            while n + length in num_set:
                length += 1
            longest = max(longest, length)

    return longest

if __name__ == "__main__":
    tests = [
        ([100, 4, 200, 1, 3, 2], 4),
        ([0, 3, 7, 2, 5, 8, 4, 6, 0, 1], 9),
        ([], 0),
        ([1, 2, 0, 1], 3),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        got = longest_consecutive(inp)

```

```
status = "PASS" if got == expected else "FAIL"
print(f"Test {i}: {status} (got {got}, expected {expected})")
```

07. Subarray Sum Equals K

07_subarray_sum_equals_k.py

Explanation

Problem: Given an integer array `nums` and integer `k`, return the total number of contiguous subarrays whose sum equals `k`. Array can contain negative numbers (rules out a simple sliding window).

Source : LeetCode 560 - Subarray Sum Equals K

Example:

Input: `nums = [1, 1, 1]`, `k = 2`

Output: 2 (subarrays `[1,1]` at indices `[0,1]` and `[1,2]`)

Idea: let `prefix[i] = sum(nums[0..i])`. A subarray `(j, i]` sums to `k` exactly when `prefix[i] - prefix[j] == k`, i.e. `prefix[j] == prefix[i] - k`. So while scanning and building the running prefix sum, count how many *earlier* prefix sums equal (current prefix - `k`) using a dict of prefix-sum -> occurrence count. Seed the dict with `{0: 1}` to correctly count subarrays that start at index 0.

Code

```
def subarray_sum(nums: list[int], k: int) -> int:
    prefix_counts = {0: 1}
    running_sum = 0
    total = 0

    for n in nums:
        running_sum += n
        total += prefix_counts.get(running_sum - k, 0)
        prefix_counts[running_sum] = prefix_counts.get(running_sum, 0) + 1

    return total

if __name__ == "__main__":
    tests = [
        ([1, 1, 1], 2, 2),
        ([1, 2, 3], 3, 2),
        ([1], 0, 0),
        ([1, -1, 0], 0, 3),
    ]

    for i, (nums, k, expected) in enumerate(tests, 1):
        got = subarray_sum(nums, k)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

08. Intersection Of Two Arrays

08_intersection_of_two_arrays.py

Explanation

Problem: Given two integer arrays, return their intersection - each element in the result must be unique, order doesn't matter.

Source : LeetCode 349 - Intersection of Two Arrays

Example:

Input: nums1 = [1, 2, 2, 1], nums2 = [2, 2]

Output: [2]

Idea: convert both arrays to sets and use set intersection directly - this is the hashing topic's version of "know when the stdlib operation already IS the optimal algorithm." Set intersection is $O(\min(\text{len}(a), \text{len}(b)))$ on average.

Code

```
def intersection(nums1: list[int], nums2: list[int]) -> list[int]:
    return list(set(nums1) & set(nums2))

if __name__ == "__main__":
    tests = [
        ([1, 2, 2, 1], [2, 2], [2]),
        ([4, 9, 5], [9, 4, 9, 8, 4], [4, 9]),
        ([], [1, 2, 3], []),
    ]

    for i, (nums1, nums2, expected) in enumerate(tests, 1):
        got = sorted(intersection(nums1, nums2))
        status = "PASS" if got == sorted(expected) else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {sorted(expected)})")
```

09. Isomorphic Strings

09_isomorphic_strings.py

Explanation

Problem: Given two strings s and t, return True if s can be transformed into t by replacing every occurrence of each character in s with another character. Mapping must be one-to-one in both directions (no two source chars map to the same target char).

Source : LeetCode 205 - Isomorphic Strings

Example:

Input: s = "egg", t = "add"

Output: True (e->a, g->d)

Input: s = "foo", t = "bar"

Output: False (o would need to map to both a and r)

Idea: a single s->t dict isn't enough - it misses the case where two different source characters both map to the same target character (mapping must be a bijection, not just a function). Keep two dicts, one per direction, and check both stay consistent at every step.

Code

```
def is_isomorphic(s: str, t: str) -> bool:
    if len(s) != len(t):
        return False

    map_st: dict[str, str] = {}
    map_ts: dict[str, str] = {}

    for cs, ct in zip(s, t):
        if cs in map_st and map_st[cs] != ct:
            return False
        if ct in map_ts and map_ts[ct] != cs:
            return False
        map_st[cs] = ct
        map_ts[ct] = cs

    return True

if __name__ == "__main__":
    tests = [
        ("egg", "add", True),
        ("foo", "bar", False),
        ("paper", "title", True),
        ("badc", "baba", False),
    ]

    for i, (s, t, expected) in enumerate(tests, 1):
        got = is_isomorphic(s, t)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

10. First Unique Character

10_first_unique_character.py

Explanation

Problem: Given a string *s*, return the index of the first character that does not repeat elsewhere in *s*. Return -1 if none exists.

Source : LeetCode 387 - First Unique Character in a String

Example:

Input: "leetcode"

Output: 0 ('l' never repeats)

Input: "loveleetcode"

Output: 2 ('v' is the first char with count 1)

Idea: two passes. First build a full frequency count (need to see the whole string before knowing which chars repeat). Second pass walks in original order and returns the first index whose character has count 1 - the dict lookup makes this $O(1)$ per character.

Code

```
def first_uniq_char(s: str) -> int:
    counts: dict[str, int] = {}
    for ch in s:
```

```

        counts[ch] = counts.get(ch, 0) + 1

    for i, ch in enumerate(s):
        if counts[ch] == 1:
            return i

    return -1

if __name__ == "__main__":
    tests = [
        ("leetcode", 0),
        ("loveleetcode", 2),
        ("aabb", -1),
        ("", -1),
        ("z", 0),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        got = first_uniq_char(inp)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```